Table of contents

Gaussian Mixture Models and the EM Algorithm

Introduction

What is a Probabilistic Model?

A **probabilistic model** is a mathematical representation of a random phenomenon. Unlike a deterministic model that always gives the same result for the same inputs, a probabilistic model describes the **probabilities** of different possible outcomes.

Why Model Data Density?

So far, we have often used models that assume data follows a specific distribution (like the normal distribution). For example:

- **In Principal Component Analysis (PCA)**: We assume data is centered and study its variance
- **In Linear Discriminant Analysis (LDA)**: We compare variance within classes and between classes

But what if we don't know what distribution our data follows? This is where **density estimation** comes in.

Density Estimation

Definition

Density estimation consists of constructing an estimate of the probability density function from an observed dataset.

If you imagine a histogram, density estimation consists of finding the smooth curve that best describes the shape of that histogram.

Density Estimation Methods

Several approaches exist:

1. **Histograms**: The simplest method, but not very accurate
2. **Parzen windows (or kernels)**: A generalization of histograms
3. **K-nearest neighbors (kNN)**: Estimates density based on distance to k nearest neighbors
4. **Mixture models**: Combines several simple distributions to model a complex distribution

Mixture Models

Fundamental Idea

The idea behind **mixture models** is simple but powerful: instead of trying to model a complex distribution with a single simple function, we model it as a combination (a "mixture") of several simple distributions.

Analogy: Imagine you have a population composed of several subgroups (for example, men and women, or young and elderly people). Instead of trying to describe the entire population with a single height curve, it would be more accurate to have one curve for each subgroup, then combine them.

Mathematical Formulation

A mixture model is written as:

$$f(x) = \sum_{j=1}^{c} \pi_j \phi(x, \theta_j)$$

Let's break down this formula:

- $f(x)$: The probability density at point x (what we want to estimate)
- c: The number of components in the mixture
- π_j: The weight of component j (the proportion of the mixture)
- $\phi(x, \theta_j)$: The density of component j at point x
- θ_j: The parameters of component j

Important constraint: The weights π_j must satisfy $\sum_{j=1}^{c} \pi_j = 1$ and $\pi_j \geq 0$ for all j.

Gaussian Mixture Model (GMM)

Why Gaussians?

The **Gaussian** (or normal) distribution is a natural choice for several reasons:

1. **Central Limit Theorem**: Many natural phenomena approximately follow a Gaussian distribution
2. **Mathematical properties**: Gaussians have very convenient analytical properties
3. **Flexibility**: A mixture of Gaussians can approximate any distribution with enough components

Formal Definition

A **Gaussian Mixture Model** with c components is written as:

$$f(x) = \sum_{j=1}^{c} \pi_j \mathcal{N}(x|\mu_j, \Sigma_j)$$

where $\mathcal{N}(x|\mu_j, \Sigma_j)$ is the density of a multivariate Gaussian distribution:

$$\mathcal{N}(x|\mu_j, \Sigma_j) = \frac{1}{(2\pi)^{D/2}|\Sigma_j|^{1/2}} \exp\left(-\frac{1}{2}(x-\mu_j)^T\Sigma_j^{-1}(x-\mu_j)\right)$$

GMM Parameters

A GMM has three types of parameters:

1. **Weights** π_j: The proportion of each component
2. **Means** μ_j: The center of each Gaussian
3. **Covariance matrices** Σ_j: The shape and orientation of each Gaussian

Probabilistic Interpretation

Latent Variables

An important interpretation of GMM is in terms of **latent variables**. A latent variable is a variable that we don't observe directly, but that influences the data we observe.

In a GMM, we can imagine the data generation process:

1. **Step 1**: Choose a component j with probability π_j
2. **Step 2**: Generate a point x from the Gaussian $\mathcal{N}(\mu_j, \Sigma_j)$

The component chosen in step 1 is our **latent variable**: we don't know which component generated each point, but this information would be very useful for estimating parameters.

Formulation with Latent Variables

If we denote z as the latent variable indicating the component ($z = j$ means "the point comes from component j"), then:

- Prior probability: $\mathbb{P}(z = j) = \pi_j$
- Conditional probability: $\mathbb{P}(x|z = j) = \mathcal{N}(x|\mu_j, \Sigma_j)$

The marginal density (what we observe) is then:

$$f(x) = \sum_{j=1}^{c} \mathbb{P}(z = j)\mathbb{P}(x|z = j) = \sum_{j=1}^{c} \pi_j \mathcal{N}(x|\mu_j, \Sigma_j)$$

Parameter Estimation: The Challenge

Intuitive Approach

If we knew the latent variables (if we knew which point comes from which component), parameter estimation would be simple:

- To estimate μ_j: Take the mean of points from component j
- To estimate Σ_j: Calculate the covariance of points from component j
- To estimate π_j: Calculate the proportion of points in component j

But the problem is precisely that we **don't know** the latent variables!

Maximum Likelihood Estimation (MLE)

The standard approach for estimating parameters is **Maximum Likelihood Estimation** (MLE). The idea is to choose the parameters that make the observed data most probable.

For N independent data points $x_1, ..., x_N$, the likelihood is:

$$\mathcal{L}(\theta) = \prod_{i=1}^{N} f(x_i|\theta) = \prod_{i=1}^{N} \sum_{j=1}^{c} \pi_j \mathcal{N}(x_i|\mu_j, \Sigma_j)$$

For practical reasons, we work with the **log-likelihood**:

$$\log \mathcal{L}(\theta) = \sum_{i=1}^{N} \log \sum_{j=1}^{c} \pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j)$$

The Technical Problem

Look at the log-likelihood formula:

$$\log \mathcal{L}(\theta) = \sum_{i=1}^{N} \log \left(\sum_{j=1}^{c} \pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j) \right)$$

The problem is the **logarithm of a sum**. If the logarithm were inside the sum, we could easily maximize. But here it's the opposite: we have a sum inside the logarithm, which makes optimization difficult.

This is the problem that the EM algorithm will solve.

Expectation-Maximization (EM) Algorithm

The Brilliant Idea

The key idea of the EM algorithm is: instead of trying to directly maximize the log-likelihood (difficult), we will maximize a simpler auxiliary function.

How? By using an **iterative two-step approach**:

1. **E-step (Expectation)**: Given the current parameters, estimate the latent variables
2. **M-step (Maximization)**: Given the estimates of the latent variables, update the parameters

And we repeat these two steps until convergence.

The Concept of Responsibility

The heart of the EM algorithm for GMM is the concept of **responsibility**.

The responsibility γ_{ij} is the **probability** that point x_i was generated by component j, given the current parameters.

Mathematically:

$$\gamma_{ij} = \mathbb{P}(\text{component } j | x_i, \theta) = \frac{\pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j)}{\sum_{k=1}^{c} \pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)}$$

Intuitive interpretation: Imagine that each data point is a person in a crowd, and each Gaussian component is a group. The responsibility γ_{ij} answers the question: "If I see this person x_i, what is the probability that they belong to group j?"

Properties of Responsibilities

1. **Normalization**: For each point i, the sum of responsibilities over all components equals 1:

$$\sum_{j=1}^{c} \gamma_{ij} = 1$$

   This means each point is "distributed" among the different components.

2. **Continuity**: $\gamma_{ij} \in [0, 1]$, unlike binary indicators $\{0, 1\}$ used in algorithms like K-means.

3. **Distance sensitivity**: A point close to the center of a component will have high responsibility for that component. A point equidistant from two components will have similar responsibilities for both.

The EM Algorithm in Detail

Step 0: Initialization

We start with an initial estimate of parameters $\theta^{(0)} = \{\pi_j^{(0)}, \mu_j^{(0)}, \Sigma_j^{(0)}\}_{j=1}^{c}$.

Common initialization strategies:

- Initialize weights uniformly: $\pi_j^{(0)} = 1/c$
- Choose means $\mu_j^{(0)}$ randomly from the data
- Initialize covariances to the identity matrix: $\Sigma_j^{(0)} = I$

E-step: Calculating Responsibilities

At iteration t, with parameters $\theta^{(t)}$, we calculate:

$$\gamma_{ij}^{(t)} = \frac{\pi_j^{(t)} \mathcal{N}(x_i|\mu_j^{(t)}, \Sigma_j^{(t)})}{\sum_{k=1}^{c} \pi_k^{(t)} \mathcal{N}(x_i|\mu_k^{(t)}, \Sigma_k^{(t)})}$$

For each point i and each component j.

Why is this step called "Expectation"? Because we calculate the expectation (the expected average value) of the latent variables given the data and current parameters.

M-step: Updating Parameters

Now that we have the responsibilities, we update the parameters:

1. **New weights**:

$$\pi_j^{(t+1)} = \frac{1}{N} \sum_{i=1}^{N} \gamma_{ij}^{(t)}$$

   Interpretation: The new weight π_j is the average responsibility assigned to component j over all points.

2. **New means**:

$$\mu_j^{(t+1)} = \frac{\sum_{i=1}^{N} \gamma_{ij}^{(t)} x_i}{\sum_{i=1}^{N} \gamma_{ij}^{(t)}}$$

   Interpretation: The new mean μ_j is the weighted average of all points, with responsibilities γ_{ij} as weights.

3. **New covariances**:

$$\Sigma_j^{(t+1)} = \frac{\sum_{i=1}^{N} \gamma_{ij}^{(t)} (x_i - \mu_j^{(t+1)})(x_i - \mu_j^{(t+1)})^T}{\sum_{i=1}^{N} \gamma_{ij}^{(t)}}$$

   Interpretation: The new covariance Σ_j is the weighted covariance of points around the new mean.

Convergence

We repeat the E and M steps until:

- Parameters change very little between two iterations
- OR the log-likelihood changes very little
- OR we reach a maximum number of iterations

Why the EM Algorithm Works?

Geometric Intuition

Imagine you're trying to cut a pizza into slices of different sizes, but you can't see the slice boundaries. How to do it?

1. Make an initial estimate of boundaries (initialization)
2. For each piece of pizza, estimate which slice it most likely belongs to (E-step)
3. Looking at all pieces assigned to a slice, redraw the boundary of that slice (M-step)
4. Repeat until boundaries stop moving

Mathematical Justification

The EM algorithm guarantees that the log-likelihood **never decreases** at each iteration. Mathematically:

$$\log \mathcal{L}(\theta^{(t+1)}) \geq \log \mathcal{L}(\theta^{(t)})$$

This comes from an important mathematical inequality called **Jensen's inequality**.

Difference between EM and K-means

K-means: A Special Case

K-means can be seen as a **special case** of the EM algorithm for a GMM with:

1. All covariance matrices equal to $\sigma^2 I$
2. $\sigma^2 \to 0$ (variance tending to zero)
3. All weights equal: $\pi_j = 1/c$

In this limit, responsibilities become **binary indicators**: $\gamma_{ij} = 1$ if j is the nearest cluster, 0 otherwise.

Detailed Comparison

Aspect	K-means	EM for GMM
Assignment type	Hard (0 or 1)	Soft (probabilities)
Cluster shapes	Spheres only	Ellipsoids of various shapes
Probabilistic model	No	Yes
Overlap handling	Poor	Excellent
Convergence speed	Fast	Slower
Robustness to outliers	Low	Better

Practical Applications

1. Unsupervised Clustering

GMM with EM is often used for clustering when:

- Clusters have non-spherical shapes
- Clusters overlap
- We want a measure of uncertainty about cluster membership

2. Density Estimation

To model the underlying distribution of data, even when it's complex and multimodal.

3. Classification

If we interpret components as classes, we can use GMM for classification by assigning each point to the component with the highest responsibility.

4. Data Generation

Once the GMM is estimated, we can generate new realistic data:

1. Draw a component j according to weights π_j
2. Draw a point x according to $\mathcal{N}(\mu_j, \Sigma_j)$

Challenges and Solutions

1. Choosing the Number of Components

How to choose c (the number of components)?

Solutions:

- Use the **Bayesian Information Criterion (BIC)** which penalizes complex models
- Cross-validation
- Examine the evolution of likelihood as a function of c

2. Initialization

EM converges to a **local optimum**. Poor initialization can lead to poor solutions.

Solutions:

- Multiple random initializations
- Use K-means for smart initialization
- K-means++ for better dispersion of initial centers

3. Numerical Problems

When a component receives very few points, its covariance matrix can become singular (non-invertible).

Solutions:

- Add a small value to the diagonal (regularization)
- Constrain the shapes of covariance matrices

Types of Covariance Matrices

1. Spherical

$\Sigma_j = \sigma_j^2 I$: All directions have the same variance.

Number of parameters: 1 per component

2. Diagonal

$\Sigma_j = \text{diag}(\sigma_{j1}^2, ..., \sigma_{jD}^2)$: Each dimension has its own variance, but no correlations.

Number of parameters: D per component

3. Full

Σ_j: Full symmetric positive definite matrix.

Number of parameters: $D(D+1)/2$ per component

Trade-off: The more constrained the matrix, the fewer parameters to estimate (good when we have little data), but the less flexible the model.

Complete Example: From Theory to Practice

Problem

We have data on animal heights and weights. We suspect there are two different species, but we don't know which data corresponds to which species.

Solution with GMM and EM

1. **Modeling**: We choose a GMM with 2 components ($c = 2$)
2. **Initialization**:
   - $\pi_1^{(0)} = \pi_2^{(0)} = 0.5$
   - $\mu_1^{(0)}$ and $\mu_2^{(0)}$ chosen randomly from the data
   - $\Sigma_1^{(0)} = \Sigma_2^{(0)} = I$
3. **First iteration (E)**: Calculate initial responsibilities
   - For a point of height 50 cm and weight 20 kg: $\gamma_{i1} = 0.7$, $\gamma_{i2} = 0.3$
   - For a point of height 100 cm and weight 80 kg: $\gamma_{i1} = 0.1$, $\gamma_{i2} = 0.9$
4. **First iteration (M)**: Update parameters
   - $\pi_1^{(1)} = 0.6$, $\pi_2^{(1)} = 0.4$ (more points resemble the first species)
   - $\mu_1^{(1)}$ becomes the weighted average of points "similar to species 1"
   - $\mu_2^{(1)}$ becomes the weighted average of points "similar to species 2"
5. **Subsequent iterations**: Repeat until convergence

6. **Final result**:

    - Two well-separated Gaussians
    - For each point, a probability of belonging to each species
    - Possibility to classify points according to the most probable species

Conclusion

Gaussian Mixture Models and the EM algorithm form a powerful duo for unsupervised learning. Their strength lies in:

1. **Flexibility**: Ability to model complex distributions
2. **Probabilistic interpretation**: Provides not only clusters but also uncertainty measures
3. **Solid theoretical foundation**: Based on rigorous statistical principles

The concept of **responsibility** is particularly important: it transforms a difficult problem (clustering with uncertainty) into an elegant solution where each point partially participates in all clusters, with weights that guide learning.

This approach illustrates a fundamental idea in machine learning: sometimes, to solve a difficult problem, we need to **introduce additional variables** (latent variables) that simplify the problem, even if these variables are not directly observable.